

AI INFRA · OPEN SOURCE BOOTCAMP

算子开发实战营 基于国产异构计算平台的全链路实践

基于AI Agent实现算子的编写、验证与优化

AI Infra开源社区 · GitLink开源平台 · DataWhale开源学习社区 · 启悟学习社区 · 九源社区 · 沐曦股份 联合推出

Patch $\neq$ Delivered

候选修改还需要上下文、约束和证据

```
def schedule_decode_step(waiting, running, kv_pool):  
-   batch = fifo_batch(waiting, running)  
+   batch = continuous_batch(waiting, running, kv_pool)  
    return run_decode(batch)
```

这段代码还没有回答：

- ◆ Prefill / Decode 混合是否安全？
- ◆ KV Block 分配与复用正确吗？
- ◆ 取消、超时、OOM 如何处理？
- ◆ 公平性和优先级是否满足 SLO？
- ◆ Benchmark 是否公平且稳定？
- ◆ 显存和其他 Workload 是否回退？

AI 生成的首先是候选修改

Candidate Patch

不是 Delivered Software

课程地图

Paradigm → Loop → Context → Harness → Skills → Eval



任务变大 → 需要循环 → 需要上下文 → 需要 Harness → 需要 Skills → 需要评测治理

Predict → Modify → Complete

任务范围扩大，验证责任随之增加

01 Predict

局部代码 → Token /
Function

人持续控制节奏

02 Modify

明确指令 → Patch /
Multi-file Edit

人审查修改结果

03 Complete

Issue + Repo + Env →
Verified Change / PR

人定义目标、边界和交付

变化不是“补全消失”，而是任务范围扩大后，系统需要承担更多验证责任。

五种 AI 编程形态

产品名不是范式定义

Inline Completion

局部上下文
→ 代码建议

Chat / Ask

问题与代码
→ 解释和方案

Multi-file Edit

明确修改指令
→ Patch

Local Agent

任务 + 本地仓库
→ 执行与测试

Delegated Agent

Issue + 远程环境
→ PR / Branch

不要问“哪个产品”，先问：任务多大？能否改变环境？人在哪里接管？

任务范围 × 自治程度

越靠右上，越需要边界与评测

右上角不是“更可靠”
而是“更需要边界和评测”



Local vs. Delegated Agent

运行环境改变，责任边界也改变

Local Agent

- ◆ 当前工作区
- ◆ 人可实时观察和中断
- ◆ 本地 Diff / Commit
- ◆ 风险集中在本机环境

Delegated Agent

- ◆ 远程隔离环境
- ◆ 后台执行，阶段性汇报
- ◆ Branch / Pull Request
- ◆ 更依赖目标、权限和验收

PR 是候选交付界面，不是“自动完成”的证明。

工程控制点

不是所有任务都该一句话全自动

Direct

小型阈值 / 配置修改

Plan-first

— 先说明阻塞与内存假设 —

Spec-driven

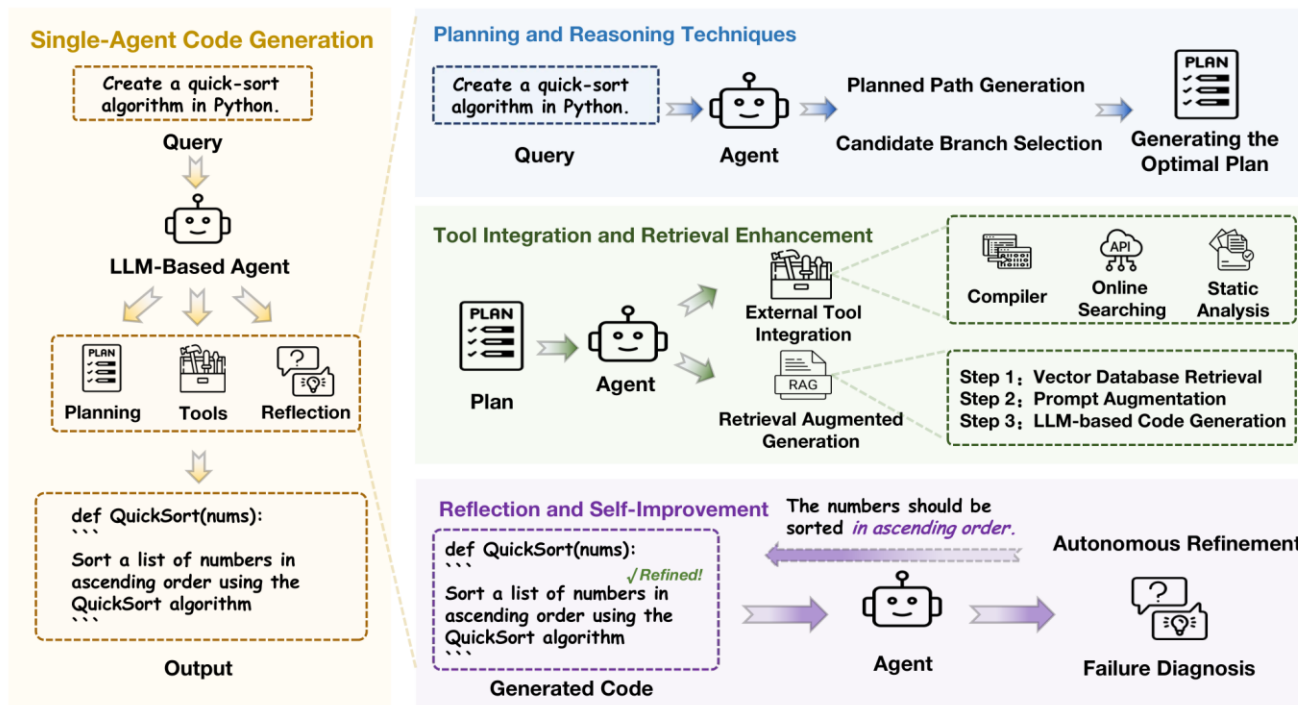
固定 SLO / 内存预算 / Fallback

Test-driven

— 先定义仿真与回归场景 —

Review-driven

审调度策略与性能证据



工程化不是让 Agent 自行猜调度策略，而是把目标、边界和证据固定清楚。

Figure source: Dong et al., A Survey on Code Generation with LLM-based Agents, Fig. 3

Multi-Agent Systems

分工带来价值，也带来协调成本

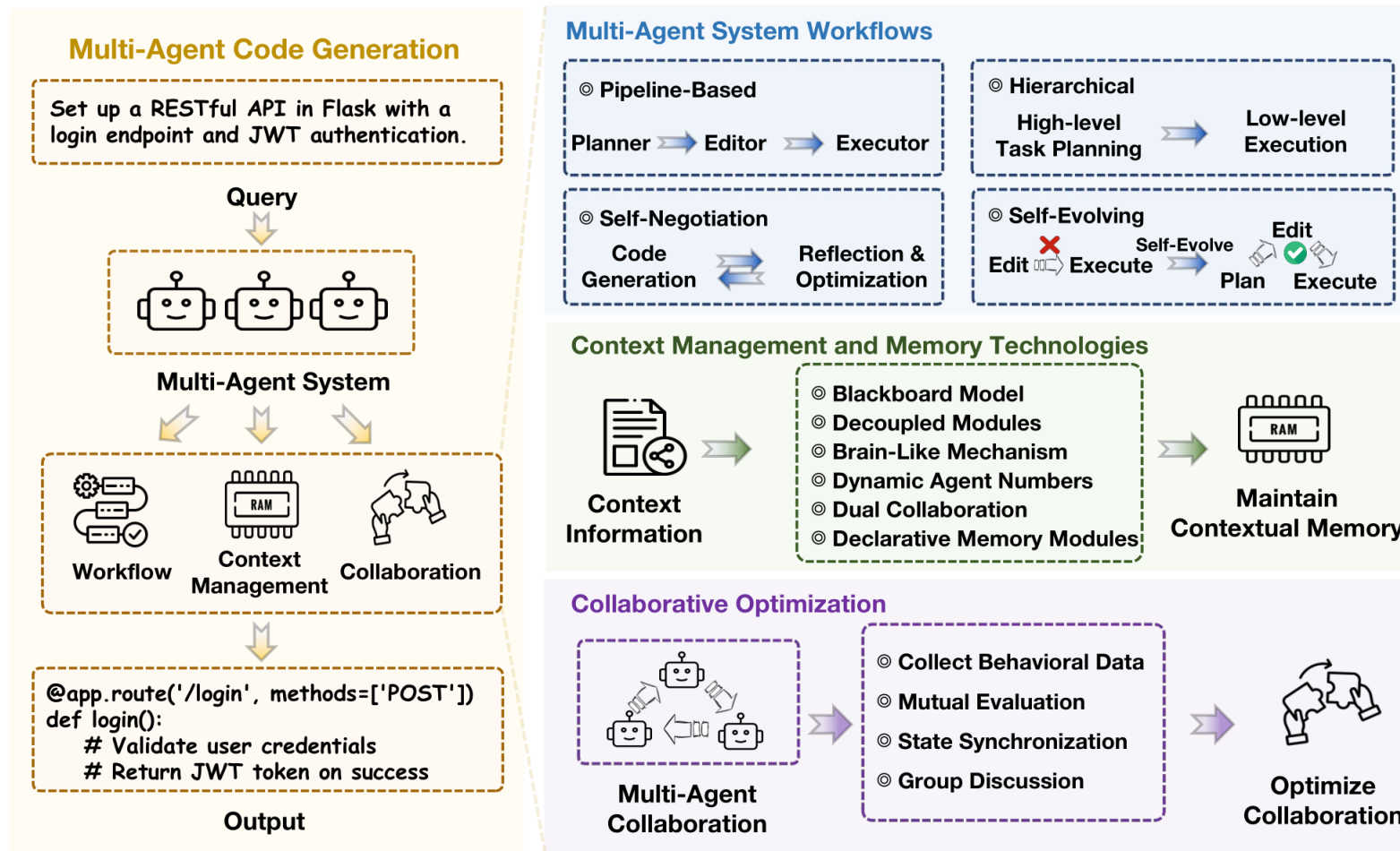


Figure source: Dong et al., A Survey on Code Generation with LLM-based Agents, Fig. 4

解释

- ◆ Workflow: 任务如何拆解和流转
- ◆ Context: 每个 Agent 看到什么
- ◆ Collaboration: 结果如何协调与合并

分工带来价值；协调带来成本。

Model $\neq$ Agent

Agent 是模型周围的一整套工程系统

MODEL $\neq$ AGENT

Agent = Model + Context + Harness + Tools + Runtime + Verification

同一模型在不同 Harness、工具和仓库环境中，表现可能完全不同。

Code Model 的最低必要原理

预测、补洞、结构化行动

预测下一批 Token

当前上下文 → 后续内容

Fill-in-the-Middle

Prefix + Suffix → Missing Code

结构化工具调用

Context + Tool Schema → Tool Call

模型只是提出“下一步做什么”；外部系统决定“能不能做、怎样做、结果是什么”。

Model / Harness / Environment

谁决策，谁执行，谁产生真实结果

Model

判断状态
选择下一步
生成参数

Harness

装配上下文
校验并执行
控制权限与停止

Environment

文件 / Shell
Git / Tests
数据库 / 服务

模型决定下一步；Harness 让这一步发生；环境给出真实结果。

Coding Agent Loop

行动—反馈—再行动

Agent Loop

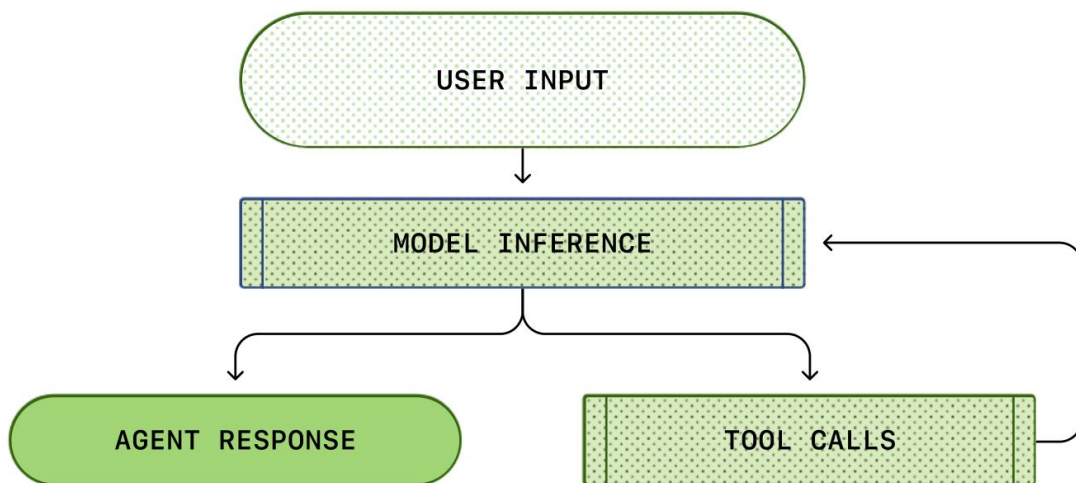


Figure source: OpenAI, Unrolling the Codex Agent Loop

看图提示

- ◆ 模型可能直接回答，也可能请求工具
- ◆ 工具结果会回到下一轮推理
- ◆ 一次任务内部包含多轮证据循环

Action → Evidence → Next Action

Context Assembly

模型每轮看到的不是用户 Prompt 本身

一次模型调用的上下文堆栈

System / Safety Instructions

Project Instructions

User Goal & Acceptance Criteria

Relevant Code & Docs

Tool Schemas

Tool Results / Test Errors / Git Diff

Skill Metadata or Loaded Skill

Context 的问题不是“有没有信息”，而是“这一轮是否最相关、最可信、最有助于下一步判断”。

Tool Calling

工具接口定义 Agent 的行动空间

```
run(command: string)
→ mixed logs / benchmark / GPU metrics
```

通用但难控制

```
run_scheduler_sim(workload)
run_kv_regression(cases)
run_benchmark(config)
collect_metrics(p50,p99,mem)
```

明确且可恢复

工具接口定义 Agent 的行动空间：仿真、回归、Benchmark 和指标要结构化返回。

六层系统架构

决策、执行、控制三段式理解

Decision

Model

Context

Execution

Harness

Tools

Control

Runtime / Permissions

Verification

六层不用死记，记住“决策—执行—控制”三段，就能重新画出来。

Continue / Stop / Escalate

成熟 Agent 不只是会做，也要会停

值得继续的条件

- ◆ 获得新证据
- ◆ 环境状态发生变化
- ◆ 有新的可行策略
- ◆ 仍在预算和权限边界内

否则

Stop → Persist State → Escalate

停止不是失败后的补救，
而是可靠 Agent 的设计组成。

成熟的 Agent 不只是会做，也要会安全地停下来。

Prompt $\neq$ Context

从指令写法到每轮信息策展

Prompt engineering vs. context engineering

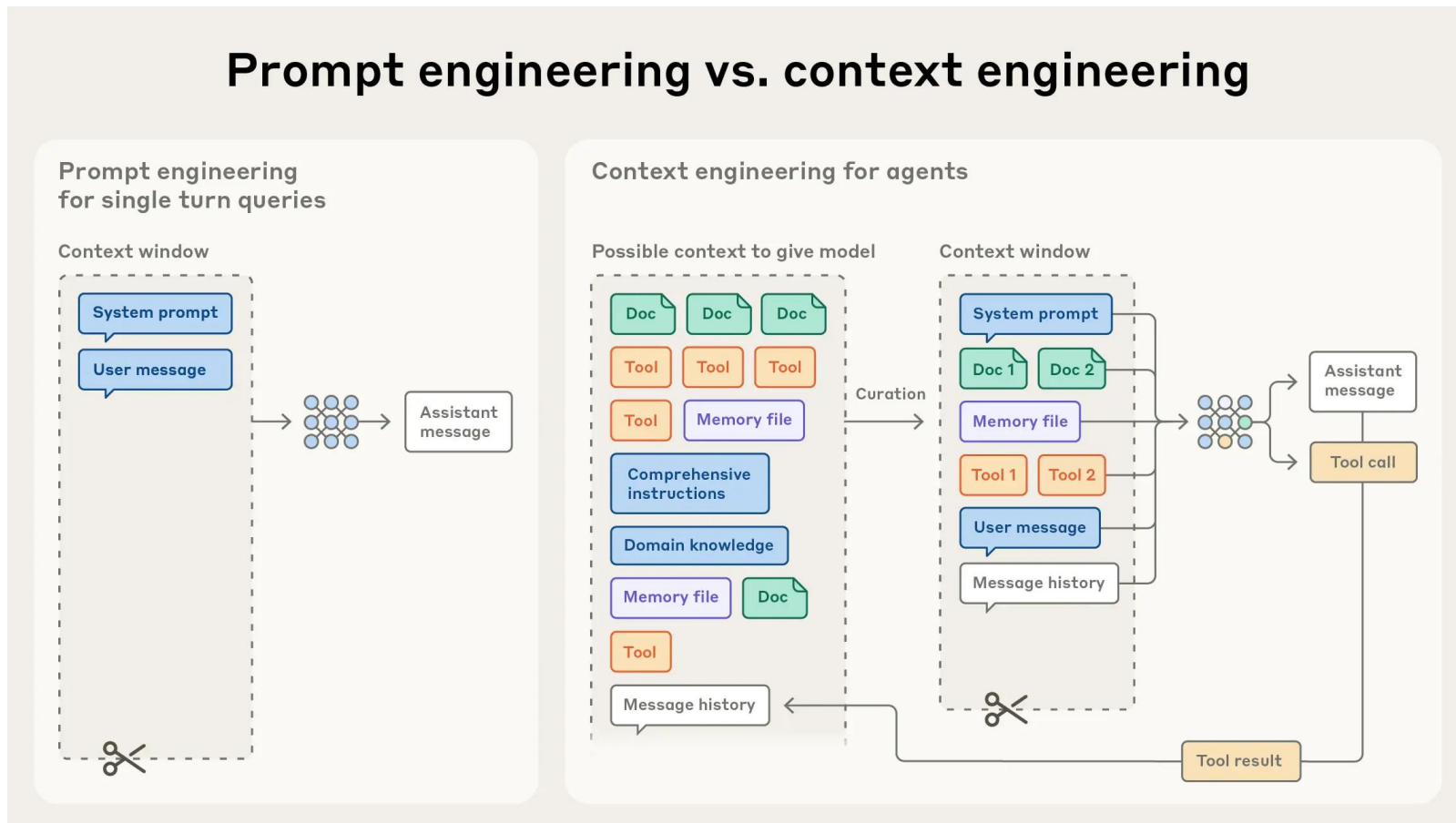


Figure source: Anthropic, Effective Context Engineering for AI Agents

看图提示

- ◆ 左边：一次性组织 Prompt
- ◆ 右边：Agent 每轮策展 Context
- ◆ 工具、记忆、历史和外部信息都会进入判断

Prompt 是 Context 的一部分，不是全部。

上下文五类来源

按来源和生命周期管理上下文

01 静态 架构 / 规范 / 构建命令

02 任务 目标 / 验收 / 非目标

03 检索 代码 / 符号 / 调用路径

04 反馈 测试 / Lint / Diff

05 状态 进度 / 决策 / 下一步

把信息分清来源，才知道它什么时候有效、应该保留多久。

Context Lifecycle

Discover → Select → Load → Use → Compress → Persist



目标不是 Maximum Context, 而是 Relevant Context。

长任务与外部状态

摘要记录进度，环境保留事实

Long Trace

- ◆ 旧日志
- ◆ 失败尝试
- ◆ 过期假设
- ◆ 无关输出

Focused State

- ◆ 目标
- ◆ 已完成
- ◆ 关键决策
- ◆ 验证证据
- ◆ 开放问题
- ◆ 下一步

状态文件回答“做到哪里了”；源码和环境回答“事实是什么”。

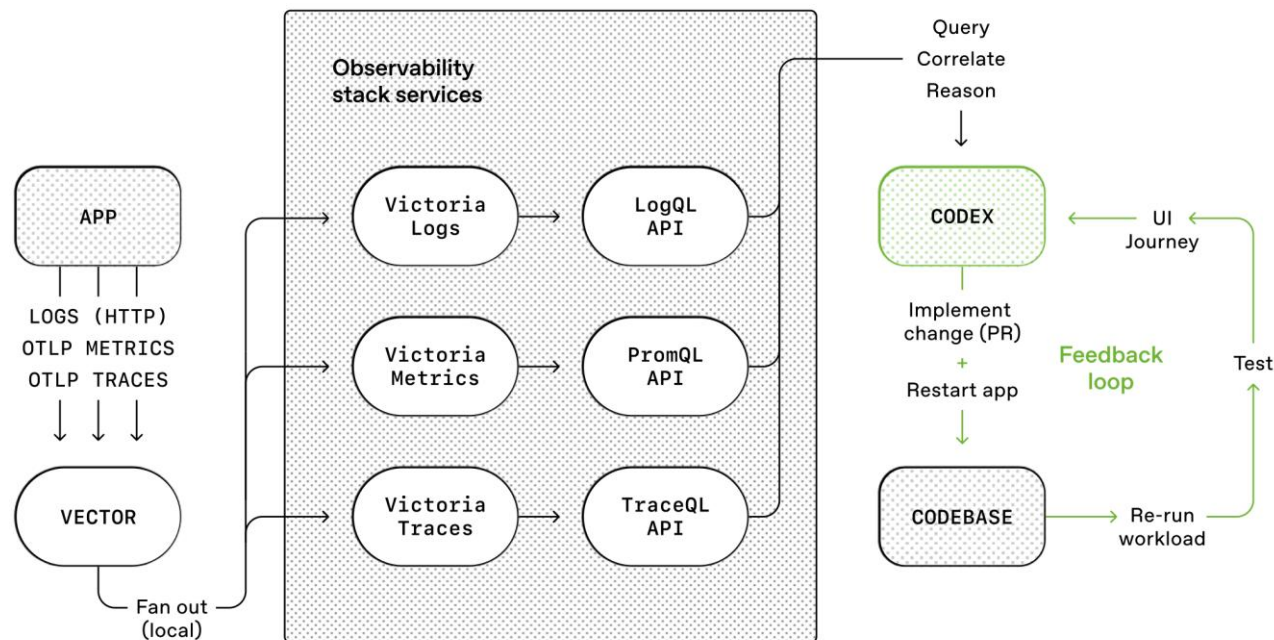
Harness Engineering

指导、感知、行动、边界

看图提示

- ◆ Sensors 不只有测试和 Lint
- ◆ 日志、指标和 Trace 是运行时反馈
- ◆ Agent 可用它们验证真实行为

Giving Agent a full observability stack



Harness 是工作环境与控制系统，不是一个 Prompt。

Figure source: OpenAI, Harness Engineering — observability stack

Agent-readable Repository

仓库是 Agent 可导航的知识系统

纯文本

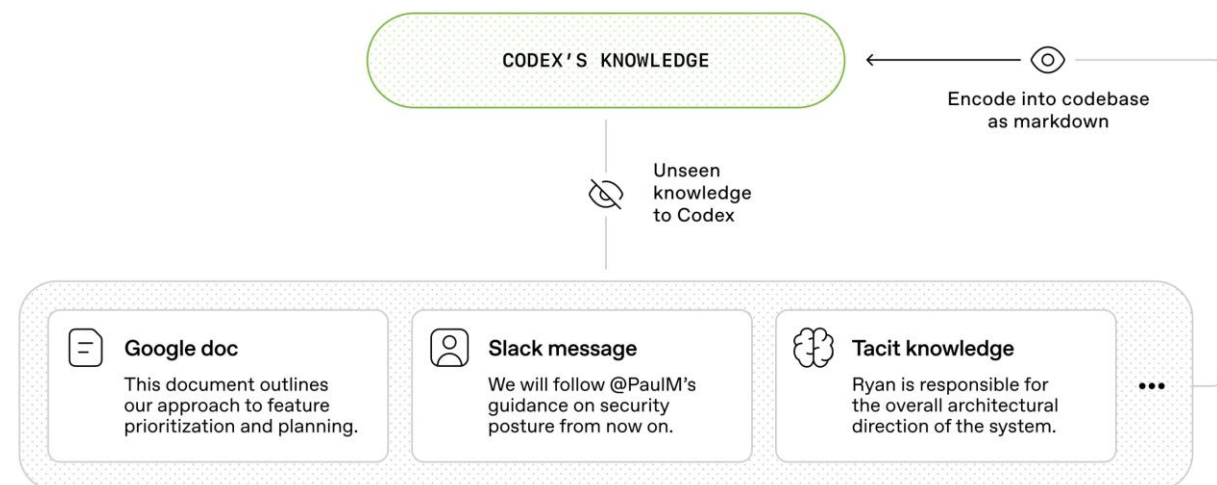
```

1 AGENTS.md
2 ARCHITECTURE.md
3 docs/
4 | design-docs/
5 | | index.md
6 | | core-beliefs.md
7 | | ...
8 | exec-plans/
9 | | active/
10 | | completed/
11 | | tech-debt-tracker.md
12 | generated/
13 | | db-schema.md
14 | product-specs/
15 | | index.md
16 | | new-user-onboarding.md
17 | | ...
18 | references/
19 | | design-system-reference-llms.txt
20 | | nixpacks-llms.txt
21 | | uv-llms.txt
22 | | ...
23 | DESIGN.md
24 | FRONTEND.md
25 | PLANS.md
26 | PRODUCT_SENSE.md
27 | QUALITY_SCORE.md
    
```

沐曦开源知识库目录



The limits of agent knowledge: What Agent can't see doesn't exist



给 Agent 一张地图，而不是一本百科全书。

Figure source: OpenAI, Harness Engineering — repository layout and limits of agent knowledge

Executable Guardrails

文档给原则，机器给反馈

Layered domain architecture with explicit cross-cutting boundaries

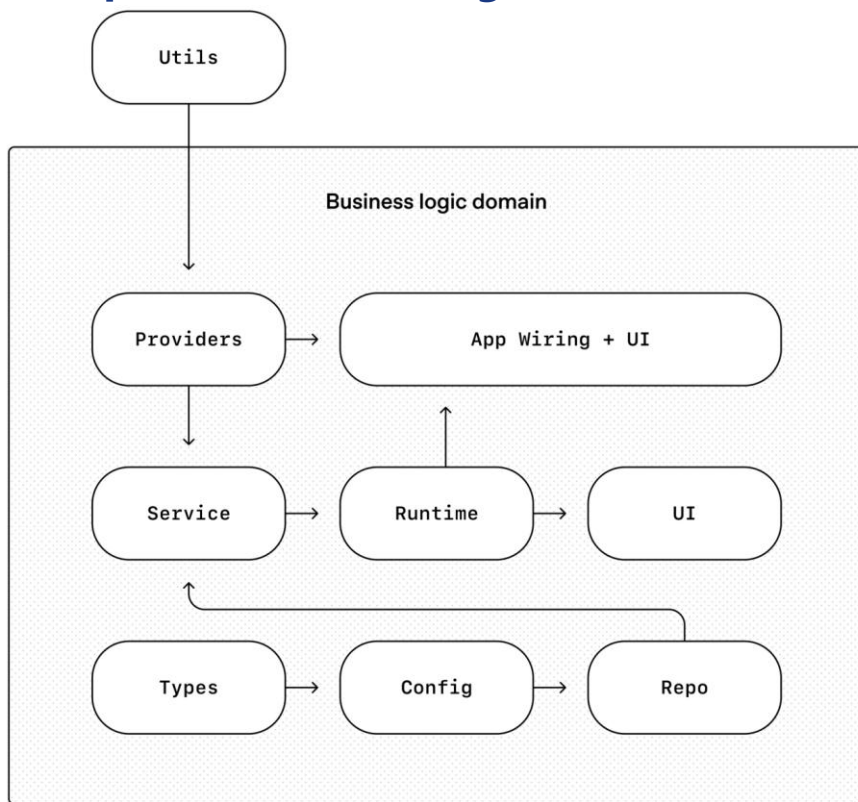


Figure source: OpenAI, Harness Engineering — layered domain architecture

映射到推理 Runtime

- ◆ Request Queue → Scheduler → KV Cache Manager → Model Runner
- ◆ Scheduler 绕过内存 / 容量检查，是越层风险

文档给原则；
架构检查给强反馈。

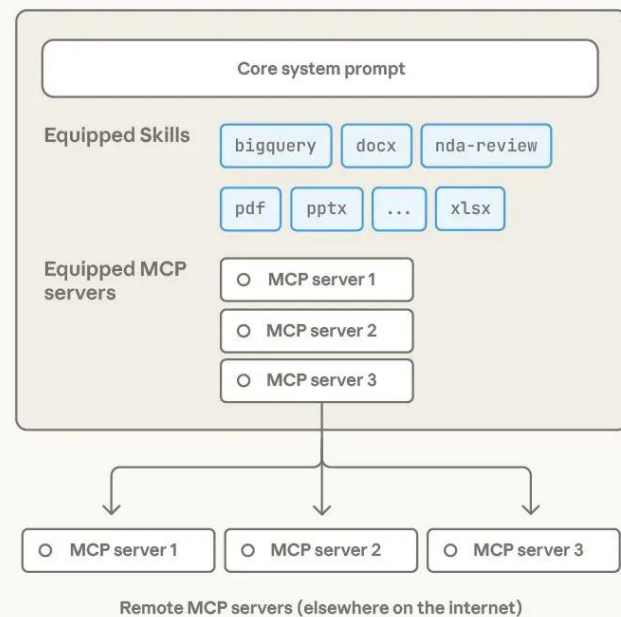
错误信息应告诉 Agent：哪个调度约束被破坏，以及应该回到哪一层修。

Agent Skills

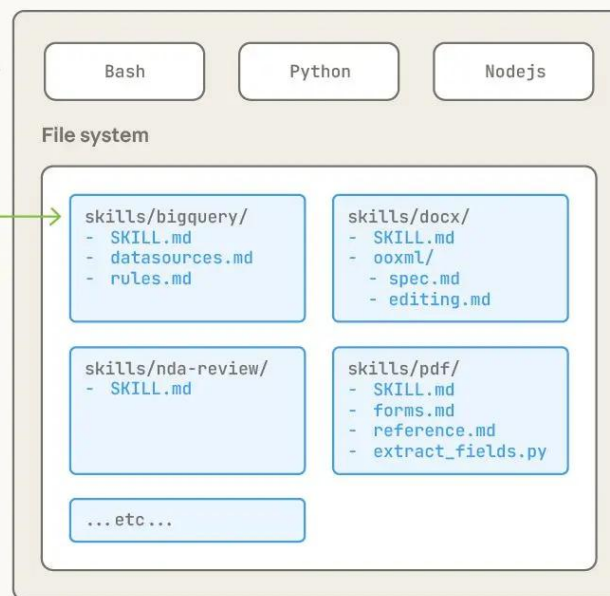
通用能力不等于团队程序性知识

Agent + Skills + Virtual Machine

Agent configuration



Agent virtual machine



看图提示

- ◆ Skill 不是模型参数
- ◆ 它通过文件系统和工具与 Agent 相连
- ◆ 补的是团队程序性知识

通用 Agent → 专用行为

Figure source: Anthropic, Equipping Agents for the Real World with Agent Skills

Skill Bundle

SKILL.md 连接脚本、参考资料和资源

Bundling additional content

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
```python
from pypdf import PdfReader, PdfWriter
```

```
Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
Extract text
text = ""
for page in reader.pages:
 text += page.extract_text()
```
```

pdf/reference.md

PDF Processing Advanced Reference

This document contains advanced PDF processing features, detailed examples, and additional libraries not covered in the main skill instructions.

pypdfium2 Library (Apache/BSD License)

Overview

pypdfium2 is a Python binding for PDFium (Chromium's PDF library). It's excellent for fast PDF rendering, image generation, and serves as ...

pdf/forms.md

If you need to fill out a PDF form, first check to see if the PDF has fillable form fields. Run this script from this file's directory:

```
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the "Fillable
fields" or "Non-fillable fields" and follow those
instructions.
```

Fillable fields

If the PDF has fillable form fields:

- Run this script from this file's directory:

```
`python scripts/extract_form_field_info.py <input.pdf>
<fields.json>`.
...
```

看图提示

- ◆ SKILL.md: 优化流程
- ◆ kv-cache-contract.md: 布局与生命周期
- ◆ scheduler-policy.md: 批处理与公平性
- ◆ benchmark-protocol.md: 性能证据
- ◆ run_regression.sh: 回归检查

主说明按需连接更多内容。

Figure source: Anthropic, Agent Skills — bundling additional content

Progressive Disclosure

丰富内容，不等于一次性全部加载

Skills and the Context Window

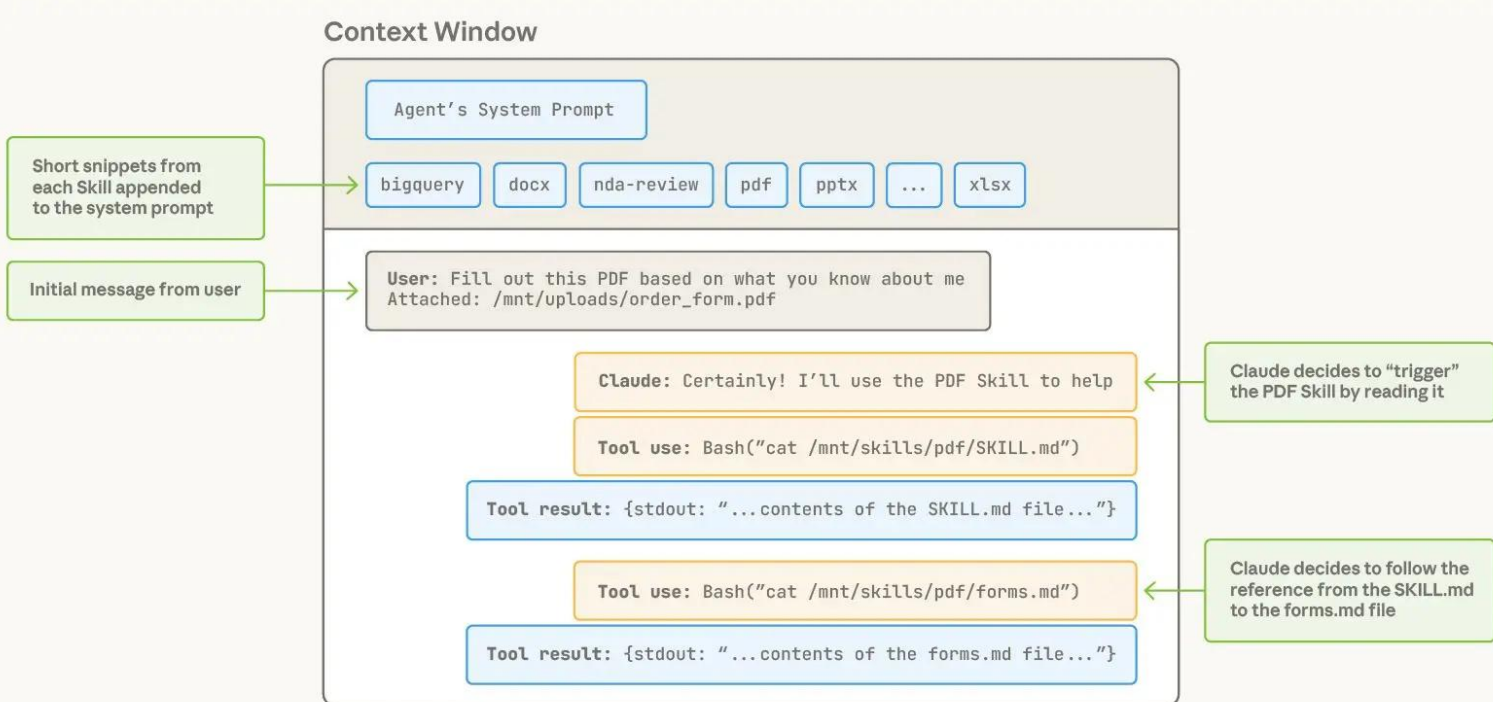


Figure source: Anthropic, Agent Skills — Skills and the Context Window

看图提示

- ◆ Metadata 始终少量存在
- ◆ 任务匹配后读取 SKILL.md
- ◆ 需要时再读 reference / script

丰富内容 ≠ 一次性全部加载

Skill Engineering

版本化、评测与供应链审查

Skill 要成为团队资产，至少经过四步

Observe

真实任务失败

Write & Version

写入并固定版本

Evaluate

有 / 无 Skill 对照

Refine

观察轨迹后迭代

Skills 的价值不在于装得多，而在于经过验证并可治理。

Agent Evaluation

不仅看结果，还要看轨迹和系统表现

Comparison: Single-Turn vs Agent Evaluations

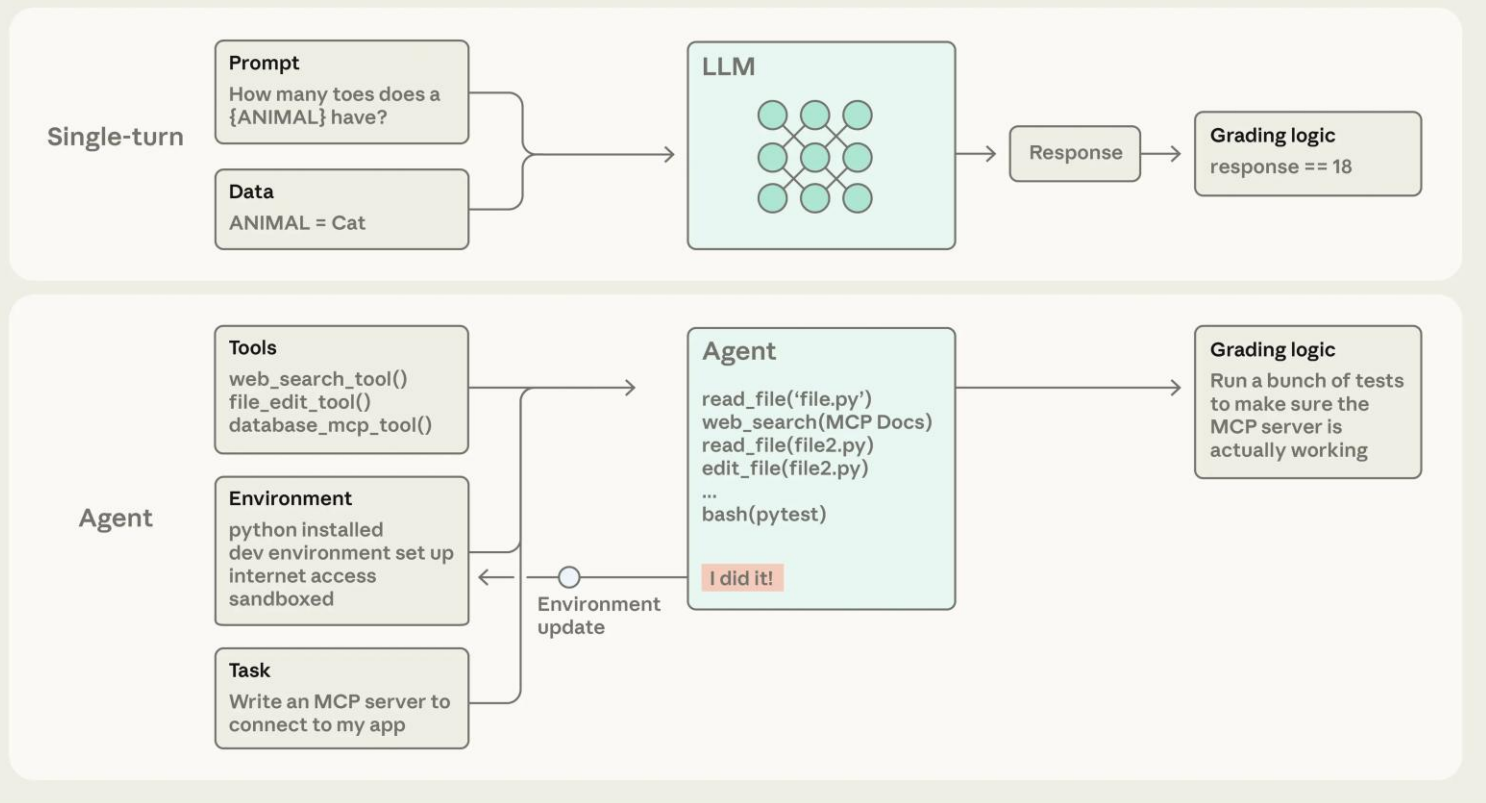


Figure source: Anthropic, Demystifying Evals for AI Agents

看图提示

- ◆ **Outcome:** 最终环境是否正确
- ◆ **Trajectory:** 过程是否合理可审计
- ◆ **System:** 多次运行是否稳定

Agent 评测的是环境变化和执行过程。

从候选修改到交付

Agent 执行, 人负责交付判断

**Candidate Scheduler Patch → Correctness & Performance
Evidence → Human Delivery Decision**

**Reliable AI Coding =
Model × Context × Harness × Skills × Verification**

Agent 可以负责大量执行工作; 人仍负责目标、边界、风险和最终交付判断。

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

阶段1：重构数据通路

你是熟悉 TileLang / GPU Kernel 的优化 Agent。
先不要修改数学公式，也不要改 GEMM 指令实现。

目标：
重构 Stage1→Stage2 的数据通路，使每个 bx 对应一个 128-token padded block，
并让中间 up_logits 按 Down GEMM 的消费 tile 存储。

约束：
1. Gate / Up / SiLU / Down / route weight 语义不变；
2. 保留实际 token 数 actual_rows 的边界正确性；
3. 先只改 tensor shape、索引和 workspace，不做 MFMA 专门化。

请输出：新 workspace 布局、索引映射、需要修改的关键代码和正确性风险。

```
- intermediate_shape = (group_sum, d_expert)
- m_start = bx*block_token - padded_offset + group_offset
- up_logits[m_start+i, by*block_dexpert+j] = value

+ workspace_k = 128
+ expert_k_tiles = d_expert // workspace_k
+ up_logits: (m_blocks, expert_k_tiles, 128, 128)
+ block_start = bx * 128
+ token_offset = block_start - group_padded_offsets[expert]
+ up_logits[bx, by, row, col] = fused_value
```

这一轮为什么先做？

- 让 producer/consumer 按同一 tile 组织数据。
- 后续 M128/M64/M32 分支可以直接写固定 workspace。

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

阶段2：优化主路径

上一轮已经完成 block/tile-major workspace。
现在只优化 Stage 1 的 full-M 主路径，暂时不要处理小 token 尾块。

目标：

- 用 TensorCoreIntrinEmitter 替代通用 T.gemm;
- 明确 M/N/K tile、warp mapping、chunk、k_pack;
- 输入 A 只从 shared/local 装载一次，同时复用给 Gate GEMM 与 Up GEMM;
- FP16 输入、FP32 累加;
- SiLU(Gate) × Up 在写 workspace 时完成。

要求：给出 emitter 参数、shared/local 布局、ldmatrix/mma 调用顺序，并说明寄存器和 shared memory 风险。

本轮性能假设：显式控制 MFMA 几何，减少通用 GEMM 的不可控映射。

关键复用：Gate 与 Up 共用输入 A，A tile 不需要重复装载。

```
- T.gemm(input_shared, gate_shared,
gate_logits_local, transpose_B=True)
- T.gemm(input_shared, up_shared,
up_logits_local, transpose_B=True)

+ stage1_mma = TensorCoreIntrinEmitter(
+     block_row_warps=4, block_col_warps=2,
+     warp_row_tiles=32, warp_col_tiles=64,
+     chunk=64, k_pack=2)
+ T.copy(X_tile, input_tile, coalesced_width=8)
+ stage1_mma.ldmatrix_a(a_local0, input_tile, 0)
+ stage1_mma.ldmatrix_b(b_local, weight_tile, 0)
+ stage1_mma.mma(a_local0, b_local,
gate_acc_local)
+ # A local registers are reused for Up GEMM
+ stage1_mma.mma(a_local0, b_local, up_acc_local)
+ up_logits[...] = up*g / (1 + exp2(-g*scale))
```

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

阶段3：处理 Stage 1 尾块

Stage 1 的 full-M MFMA 主路径已经完成。
现在只优化 `actual_rows <= 64` 的 token tail，不能让大块路径变慢。

请基于 `actual_rows` 设计 shape dispatch:

1. `actual_rows > 64`: 继续走 M128×N128 主路径;
2. `actual_rows <= 32`: 设计 true-M32 emitter, 并尽量复用同一份 A;
3. 33~64: 设计 M64 tail, 并考虑一次打包多个 N tile;
4. 所有分支写入同一个 4D workspace 格式。

请解释每个分支的 warp geometry、为什么能减少空算，以及新增分支的维护成本。

```
- # one fixed M=128 path + only boundary checks
- for i,j in T.Parallel(128, block_n):
-     if i < actual_rows: store(...)

+ if actual_rows > 64:
+     # full M128 × N128 path
+     stage1_mma = 4×2 warps, warp tile 32×64
+ elif actual_rows <= 32:
+     # true-M32 path
+     stage1_m32_mma = 1×8 warps, warp tile 32×16
+     # reuse A while covering two N128 slabs
+ else:
+     # M64 tail
+     tail_mma = 2×4 warps, warp tile 32×16
+     gate_pack = 4    # process 4 × N64 packs
```

Agent判断原则：只为低利用率 shape 做专门化；大块主路径保持简单。

MoE 不规则性：Expert token 数天然不均匀。

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

阶段4：优化 Down GEMM

Stage 1 已经完成主路径与 tail specialization。
现在只优化 Down GEMM，不修改 Stage 1。

请分析两类 workload：

- hidden-dominant: $3 \cdot d_{\text{expert}} \leq d_{\text{hidden}} < 8 \cdot d_{\text{expert}}$;
- deep-K: $d_{\text{expert}} \geq 4 \cdot d_{\text{hidden}}$ 。

目标：

1. 根据 H:E 比例选择不同 down_mma warp geometry;
2. hidden-dominant 场景为 $\text{rows} \leq 16 / \leq 32 / \leq 64$ 建立三套 tail emitter;
3. route weight 尽量放入 local register，再与 accumulator 融合写回;
4. 同时给出 swizzle / occupancy 建议。

要求：每个 dispatch 条件都必须说明它针对的硬件利用率问题。

从参数调优升级：不是找一个最佳 tile，而是按 workload shape 选择一组最佳路径。

```
- T.gemm(up_logits_shared, down_w_shared,
output_local, transpose_B=True)
- T.use_swizzle(10)

+ hidden_dominant = (H >= 3*E) and (H < 8*E)
+ if E >= (H << 2):                # deep-K
+     down_mma = 4x1 warps, warp tile 32x128
+ else:
+     down_mma = 2x2 warps, warp tile 64x64
+ if hidden_dominant:
+     rows<=16 -> down_m16_mma
+     rows<=32 -> down_tail_mma
+     rows<=64 -> down_m64_mma
+ gate_swizzle = 8 if hidden_dominant and
16<n_experts<64 else 32
+ down_swizzle = 32
+ T.annotate_min_blocks_per_sm(3)
```

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

阶段5：收尾

计算核心已经完成，不再修改数学和 MMA 路径。
现在只优化运行时工程开销。

请检查：

1. 相同 shape 是否会重复 JIT 编译；
2. Stage1→Stage2 workspace 是否会每次 torch.empty；
3. cache key 是否覆盖 device / dtype / shape；
4. 是否有可以开启的 predicated load/store lowering；
5. 缓存不能引入 shape 错配或跨 device 错用。

输出：kernel cache key、workspace cache key、生命周期策略、潜在显存代价，以及最终 run_kernel 调用结构。

**优化链：布局 → MFMA 主路径 → tail 专门化 →
Down dispatch → runtime cache**

```
+ _KERNEL_CACHE = {}  
+ _WORKSPACE_CACHE = {}  
  
+ workspace_key = ('v467a_workspace', device,  
m_blocks, E, dtype)  
+ if workspace_key not in _WORKSPACE_CACHE:  
+     _WORKSPACE_CACHE[key] =  
torch.empty((m_blocks,E//128,128,128), ...)  
+ up_logits = _WORKSPACE_CACHE[workspace_key]  
  
+ kernel_key = ('v467a', H, E, n_experts,  
padded_rows, raw_rows, m_blocks)  
+ if kernel_key not in _KERNEL_CACHE:  
+     _KERNEL_CACHE[key] =  
make_moe_kernel_v467a(...)  
+ _KERNEL_CACHE[kernel_key](..., up_logits, out)  
  
+ "tl.enable_lower_ldgstg_predicated": True
```

案例：优化TileLang 编写的面向 MoE 中 “Expert FFN 计算阶段” 的 GPU Kernel

曦云C500-16GB - 结果展示：

| grid | block | shm_bytes | regs_per_thread | kernel_latency_us | kernel_1_latency_us | sum_latency_us |
|----------|---------|-----------|-----------------|-------------------|---------------------|----------------|
| 34x64x1 | 256x1x1 | 65536 | 256 | 8965.848615 | 5324.150154 | 14289.99877 |
| 68x16x1 | 256x1x1 | 65536 | 256 | 16305.25046 | 8576.945231 | 24882.19569 |
| 135x16x1 | 256x1x1 | 65536 | 256 | 31141.08062 | 16860.96739 | 48002.048 |

| grid | block | shm_bytes | regs_per_thread | kernel_latency_us | kernel_1_latency_us | sum_latency_us |
|----------|---------|-----------|-----------------|-------------------|---------------------|----------------|
| 30x64x1 | 512x1x1 | 32768 | 126 | 1868.1344 | 1190.0416 | 3058.176 |
| 60x16x1 | 512x1x1 | 32768 | 126 | 3517.764267 | 1768.6528 | 5286.417067 |
| 119x16x1 | 512x1x1 | 32768 | 126 | 6689.041067 | 3412.241067 | 10101.28213 |

4.67x
4.71x
4.75x

JOIN THE COMMUNITY

一起加入 AI Infra 开源社区

课程仓库: <https://www.gitlink.org.cn/ccf-ai-infra/gollamago>



扫码加入学习社群

Issue 提交作业与提问
直播答疑 · 算力券支持